

Санкт-Петербургский государственный университет

РУБАНОВ Павел Евгеньевич

Выпускная квалификационная работа

**Разработка системы для сегментации,
контурирования и параметрического
анализа биологических объектов**

Уровень образования: бакалавриат

Направление *02.03.03 «Математическое обеспечение и администрирование
информационных систем»*

Основная образовательная программа *СВ.5162.2020 «Технологии программирования»*

Научный руководитель:
зав. кафедры информационно-аналитических систем, к.ф.-м.н. Михайлова Е. Г.

Консультант:
м.н.с. лаборатории Гербарий, ФГБУН «ГБС РАН», Шкурко А. В.

Рецензент:
аналитик, группа аудиометрических исследований, АО «Медиаскоп» Мамаева А. С.

Санкт-Петербург
2026

Saint Petersburg State University

Pavel Rubanov

Bachelor's Thesis

Development of a system for segmentation,
contouring, and parametric analysis of
biological objects

Education level: bachelor

Speciality *02.03.03 "Software and Administration of Information Systems"*

Programme *CB.5162.2020 "Programming Technologies"*

Scientific supervisor:
head of the Chair of Analytical Information Systems, C.Sc. E. G. Mikhailova

Reviewer:
analyst, audience measurement research group, "Mediascope" JSC A. S. Mamaeva

Saint Petersburg
2026

Оглавление

Введение	5
1. Постановка задачи	6
1.1. Цель и задачи работы	6
2. Обзор существующих решений	7
2.1. Инструменты для морфометрии и сегментации изображений	7
2.2. Сравнительный анализ	8
2.3. Выбор модели сегментации	8
3. Разработка приложения	13
3.1. Требования к системе	13
3.2. Стек технологий	14
3.3. Архитектура приложения	15
3.4. Интерактивная сегментация	17
3.5. Канонизация листа	20
3.6. Параметрические измерения листа	21
3.7. Автоматическая сегментация и параметрические измерения	23
3.8. Оптимизация производительности автоматической сегментации	24
3.9. Тестирование	26
3.10. Сборка и публикация	27
4. Обучение классификатора отбора листьев	28
4.1. Постановка задачи	28
4.2. Архитектура классификатора	28
4.3. Сбор обучающей выборки	29
4.4. Результаты	30
4.5. Сравнение с альтернативными архитектурами	32
5. Эксперимент и апробация	36
5.1. Цель и методология	36

5.2. Сравнение длин листьев	36
5.3. Сравнение поперечных профилей ширины	37
5.4. Итоговые показатели	38
5.5. Сравнение по затратам времени	38
5.6. Апробация	38
Заключение	40
Список литературы	41

Введение

Морфометрия — количественное описание формы биологических объектов — является одним из ключевых инструментов таксономии и экологии [2, 11]. Несмотря на развитый аппарат статистического анализа, получение исходных данных по-прежнему остаётся ручной или полуавтоматической операцией: исследователь вручную обводит контур каждого объекта в растровом редакторе и снимает измерения. При сериях из десятков и сотен образцов это занимает значительное время и вносит субъективную погрешность, особенно когда измерения ведут несколько человек.

Задача осложняется при работе с морфологически сложными объектами. Листья мхов рода *Sphagnum* — экологически значимых эдификаторов торфяных болот и тундровых сообществ [6, 23] — состоят из одного слоя клеток [9, 17], могут быть прозрачными и иметь неоднородные, рваные края. Существующие программы автоматической «обрисовки» контуров (например, tpsDIG [21]) требуют хорошо контрастированных объектов без инородных включений и не справляются с такими объектами. Дополнительно, на одном снимке обычно присутствует несколько листьев разного качества — целые, повреждённые, перекрывающиеся, нелистовые объекты, — и для измерений пригодны только так называемые «хорошие» листья: это требует от исследователя ещё и операции отбора.

С технической точки зрения проблема состоит в отсутствии инструмента, способного автоматически пройти весь конвейер обработки снимка — от выделения кандидатных объектов до отбора пригодных для измерений листьев и снятия с них морфометрических параметров. Появление foundation-моделей сегментации открывает возможность строить такой конвейер без сбора размеченной выборки контурных масок; задача отбора корректных листьев из полученного набора кандидатов при этом решается отдельной обучаемой моделью на сравнительно небольшой выборке. Настоящая работа направлена на построение такой системы.

1. Постановка задачи

1.1. Цель и задачи работы

Цель работы — разработать настольное приложение, полностью автоматизирующее цикл «загрузка снимков → сегментация всех листьев → отбор пригодных → параметрические измерения → экспорт» для морфометрии листьев *Sphagnum*. В случаях, когда полная автоматизация неприменима (например, для вида листа, по которому модель отбора ещё не обучена), приложение должно поддерживать интерактивный режим работы с возможностью ручной коррекции масок. Приложение должно работать на стандартных рабочих станциях лаборатории без выделенного GPU.

Для достижения цели поставлены следующие **задачи**:

1. Сформулировать требования к разрабатываемой системе.
2. Спроектировать архитектуру приложения и реализовать его.
3. Собрать и разметить обучающую выборку для классификатора отбора листьев.
4. Обучить модель отбора корректных листьев для нескольких видов *Sphagnum* и оценить её качество.
5. Сравнить результаты автоматических измерений с ручной разметкой биологов и провести апробацию системы.

Практическая значимость работы состоит в существенном сокращении времени, затрачиваемого на получение морфометрических данных, и в повышении воспроизводимости измерений за счёт стандартизованного алгоритма обработки.

2. Обзор существующих решений

2.1. Инструменты для морфометрии и сегментации изображений

Задача получения морфометрических данных с изображений биологических объектов решается несколькими классами инструментов.

ImageJ / FIJI [13] является стандартом де-факто в биологических лабораториях. Пакет предоставляет широкий набор инструментов для ручного обведения контуров и измерений, а также плагины для полуавтоматической пороговой сегментации (Threshold, Wand tool, Trainable Weka). Применительно к рассматриваемой задаче у FIJI выявлен ряд ограничений: высокий порог входа (большое количество меню, плагинов и параметров требует существенного опыта работы с пакетом); пороговая сегментация требует ручной настройки порогов под каждую серию снимков и не справляется с прозрачными объектами; рабочий процесс фрагментирован между плагинами.

tpsDIG2 [21] — специализированный инструмент для геометрической морфометрии, позволяющий расставлять landmarks и строить контуры (outlines). Требуется хорошо контрастированных объектов без инородных включений и неоднородных краёв. Листья *Sphagnum* с рваными краями и прозрачной текстурой плохо поддаются автоматической «обрисовке».

PlantCV [28] — открытая Python-библиотека для анализа изображений растений. Поддерживает пакетную обработку и разнообразные морфометрические функции (скелетизацию, измерение площади, периметра, длины). Не имеет графического интерфейса, требует написания скриптов и опирается на пороговые методы.

Labelme / CVAT / Label Studio — инструменты ручной разметки изображений, позволяющие строить полигональные аннотации. Предназначены для создания обучающих наборов данных, а не для морфометрии; не выполняют измерений.

2.2. Сравнительный анализ

Сводное сравнение рассмотренных инструментов по критериям, непосредственно влияющим на трудоёмкость морфометрической обработки серий гербарных снимков, приведено в табл. 1.

Таблица 1: Сравнение существующих инструментов по ключевым критериям

Инструмент	Сегм. сложных объектов без обуч.	Отбор листьев	Ручная коррекция маски	Замер длины и попереч. сечений	Стандарт. вывод	Пакетная обработка
ImageJ / FIJI	—	—	+	±	+	+
tpsDIG2	±	—	—	±	±	±
PlantCV	—	—	—	±	+	+
Labelme / CVAT	±	—	+	—	—	±
Разраб. система	+	+	+	+	+	+

+ — поддерживается; ± — частично / требует ручной настройки; — — не поддерживается

Ни один из рассмотренных инструментов не обеспечивает надёжной сегментации морфологически сложных листьев *Sphagnum* и не имеет автоматического отбора пригодных образцов; разрабатываемая система решает обе задачи (см. раздел 4.2).

2.3. Выбор модели сегментации

Ключевым техническим решением при разработке системы является выбор метода сегментации. Ниже рассмотрены три класса подходов и обоснован итоговый выбор.

Требования к моделям сегментации и отбора

Прежде чем сравнивать подходы, необходимо учесть особенности рабочего материала. На каждом снимке присутствует несколько листьев разного качества — целые, повреждённые, перекрывающиеся, — а также нелистовые объекты (фон, артефакты препарирования). Для морфометрии нужны только «хорошие» листья: целые, с неповреждёнными краями, типичной для вида формы, не перекрывающиеся с другими объектами.

Полностью автоматический конвейер обработки изображения, заявленный целью работы (см. раздел 1), распадается на два связанных подзадания:

1. **Сегментация всех объектов-кандидатов** на снимке, без указания пользователем конкретной цели.
2. **Отбор корректных листьев** из полученного набора кандидатов.

К моделям, решающим эти подзадания, предъявляются следующие требования.

Модель сегментации должна работать на произвольных листьях *Sphagnum* без обучения под конкретный вид. Сбор репрезентативной обучающей выборки контурных масок потребовал бы значительных трудозатрат самих биологов — той самой ручной работы, которую и требуется автоматизировать.

Модель отбора, напротив, обучается под конкретный вид листа: морфология «хорошего» листа существенно различается у разных видов, поэтому единый универсальный классификатор работает хуже специализированных. При этом разметка для отбора (хороший лист / плохой лист / не лист) принципиально проще, чем разметка контуров.

Дополнительно желательно, чтобы та же модель сегментации поддерживала и *интерактивный режим сегментации* — для случаев, когда классификатор отбора для нового вида ещё не обучен и обработка ведётся вручную.

Ниже рассмотрены три класса подходов к сегментации и обоснован итоговый выбор; вопрос модели отбора рассматривается отдельно в разделе 4.2.

Классические алгоритмы компьютерного зрения

Сотрудники лаборатории Гербарий ГБС РАН ранее пробовали применять для сегментации листьев классические алгоритмы (пороговая

сегментация, морфологические операции, GrabCut, watershed) в стандартных пакетах компьютерного зрения, но не достигли приемлемого качества: на прозрачных и краеломких листьях *Sphagnum* такие методы строят фрагментированные маски, ручная коррекция которых сопоставима по трудоёмкости с полной ручной разметкой. Поэтому классические методы не рассматривались в данной работе в качестве самостоятельной сегментирующей модели.

Специализированные нейросетевые модели (U-Net и аналоги)

Архитектуры типа U-Net [22] требуют размеченной обучающей выборки, сбор которой выходит за рамки настоящей работы. Кроме того, готовая foundation-модель (см. ниже) решает задачу сегментации без какого-либо дообучения, что делает обучение специализированной модели избыточным.

Foundation-модели сегментации: SAM и MobileSAM

Segment Anything Model (SAM) [26] — foundation-модель сегментации от Meta AI¹. Веса модели (checkpoint) и код инференса (Python-пакет `segment-anything`) распространяются под лицензией Apache 2.0, что допускает их свободное использование, в том числе во встраиваемых распространяемых приложениях. Ключевое свойство SAM — способность сегментировать произвольный объект *без дообучения на конкретном домене*.

SAM поддерживает оба режима работы, нужных приложению. В **автоматическом** режиме встроенный Automatic Mask Generator (AMG) равномерно расставляет точечные подсказки по сетке и для каждой строит маску, после чего фильтрует результаты по качеству и устойчивости — это даёт набор кандидатов для последующего отбора классификатором. В **интерактивном** режиме SAM строит маску по подсказкам от пользователя (точки, ограничивающий прямоугольник), что исполь-

¹Meta Platforms Inc. — организация признана экстремистской, её деятельность запрещена на территории Российской Федерации.

зуется в приложении для ручной разметки на новых видах листьев и для коррекции автоматически полученных масок.

Базовая версия SAM имеет существенное ограничение для настольного приложения: image encoder на основе ViT-H содержит 632 М параметров и требует GPU для приемлемой скорости работы.

MobileSAM [12] решает эту проблему: тяжёлый encoder заменён на компактный ViT-T ($\approx 5,7$ М параметров), обученный методом knowledge distillation на выходах оригинального SAM encoder. Размер checkpoint — 39 МБ против 2,6 ГБ у ViT-H SAM. Веса MobileSAM, код инференса и код дистилляции из оригинального SAM также опубликованы под лицензией Apache 2.0. Авторы показывают, что на задачах prompt-based сегментации качество MobileSAM сопоставимо с оригиналом [12], при этом на CPU среднего рабочего компьютера модель генерирует маску по интерактивной подсказке менее чем за секунду.

Качество масок MobileSAM на листьях *Sphagnum* проверено визуально специалистом лаборатории Гербарий ГБС РАН (м.н.с. Шкурко А. В.) на нескольких десятках снимков и признано приемлемым для дальнейшей работы.

Итог

MobileSAM выбран в качестве модели сегментации по совокупности факторов:

- не требует обучающих данных и обобщается на новые виды листьев;
- поддерживает оба режима работы приложения — автоматический (через AMG для пакетной обработки) и интерактивный (через подсказки для ручной разметки и коррекции);
- работает на CPU, не требует GPU;
- уместается в дистрибутив настольного приложения (39 МБ);

- веса и код инференса распространяются под лицензией Apache 2.0, что допускает встраивание в распространяемое приложение.

SAM и MobileSAM не имеют собственного интерфейса, средств морфометрии и модуля отбора — именно их интеграция с обучаемым классификатором отбора, интерактивной коррекцией и параметрическими измерениями в единую систему и составляет основной вклад разрабатываемой системы.

3. Разработка приложения

3.1. Требования к системе

Требования сформулированы совместно с консультантом (м.н.с. лаборатории Гербарий ГБС РАН, Шкурко А. В.) на основе технического задания.

Требования

- Автоматическое распознавание формы листа и построение полигональной маски контура.
- Определение длины и ширины полигона.
- Расстановка точек вдоль перпендикуляров к главной оси с заданным шагом.
- Просмотр исходного изображения и маски на экране.
- Возможность задать масштабный коэффициент (1 пиксель = ... мм) для вывода измерений в физических единицах.
- Экспорт результатов в табличный формат (CSV / Excel).
- Пакетная обработка серий изображений.
- Возможность ручной корректировки автоматически построенной маски.
- Режим автоматической сегментации и параметрических измерений: построение масок всех объектов без участия пользователя, автоматический отбор корректных листьев и запуск измерений.

Нефункциональные требования

- Работоспособность на стандартных рабочих станциях без выделенного GPU (инференс всех моделей на CPU).

- Время отклика модели сегментации не более 1 с на одно изображение в интерактивном режиме.
- Время обработки одного изображения не более 10 с в автоматическом режиме.
- Поддержка форматов JPEG, PNG, TIFF.
- Воспроизводимость результатов: одни и те же входные данные и подсказки дают одинаковый вывод.
- Единый формат хранения масок (PNG) и измерений (CSV / XLSX).

3.2. Стек технологий

В качестве основного языка реализации выбран Python. Для MobileSAM авторами опубликован готовый инференс-код на Python (пакет `mobile_sam` поверх PyTorch [20]); MobileNetV3-Small [25], используемая как экстрактор признаков для классификатора отбора, доступна с предобученными весами в составе библиотеки `torchvision`. Градиентный бустинг XGBoost [5] (с использованием служебных функций `scikit-learn` [24]) реализует сам классификатор отбора; алгоритм параметрических измерений построен на OpenCV [3] и NumPy [1]. Кроссплатформенный фреймворк PyQt6 покрывает все нужные приложению элементы интерфейса. Используемые библиотеки приведены в табл. 2.

Таблица 2: Используемые технологии

Компонент	Технология	Обоснование
Сегментирующая модель	MobileSAM (PyTorch)	Foundation model, CPU, 39 МБ
Экстрактор признаков	MobileNetV3-Small (torchvision)	Предобучение на ImageNet [16], 9,7 МБ
Классификатор отбора	XGBoost, scikit-learn	Градиентный бустинг, поддержка весов классов
Графический интерфейс	PyQt6	Кроссплатформенный, мощный QGraphicsView
Обработка изображений	OpenCV, NumPy	Стандарт для CV-задач
Чтение изображений	Pillow	Поддержка JPEG, PNG, TIFF
Визуализация измерений	matplotlib	Стандарт научной графики [15]
Экспорт CSV	csv (stdlib)	Совместимость с Excel
Экспорт XLSX с миниатюрами	openpyxl	Встраивание изображений в ячейки

3.3. Архитектура приложения

Приложение реализовано на языке Python с использованием фреймворка PyQt6 для графического интерфейса и библиотеки PyTorch для инференса MobileSAM. Исходный код опубликован по адресу https://github.com/pavelrubanov/segmentation_system.

Кодовая база разделена на три слоя (рис. 1):

- **Ядро (src/core/)**. Вычислительные модули, не зависящие от UI: обёртка над MobileSAM, конвейер предобработки маски и канонизации листа, алгоритм параметрических измерений, общий конвейер пакетной обработки и экспорт результатов.
- **Интерфейс (src/ui/)**. Окна и элементы интерфейса PyQt6: навигационное окно, окно интерактивной сегментации, общие компоненты пакетных диалогов и точки входа в пакетные режимы.
- **Классификатор (classifier/)**. Изолированный пакет с собственными зависимостями: фиксированный экстрактор признаков MobileNetV Small и обучаемый XGBoost (подробности — в разделе 4.2).

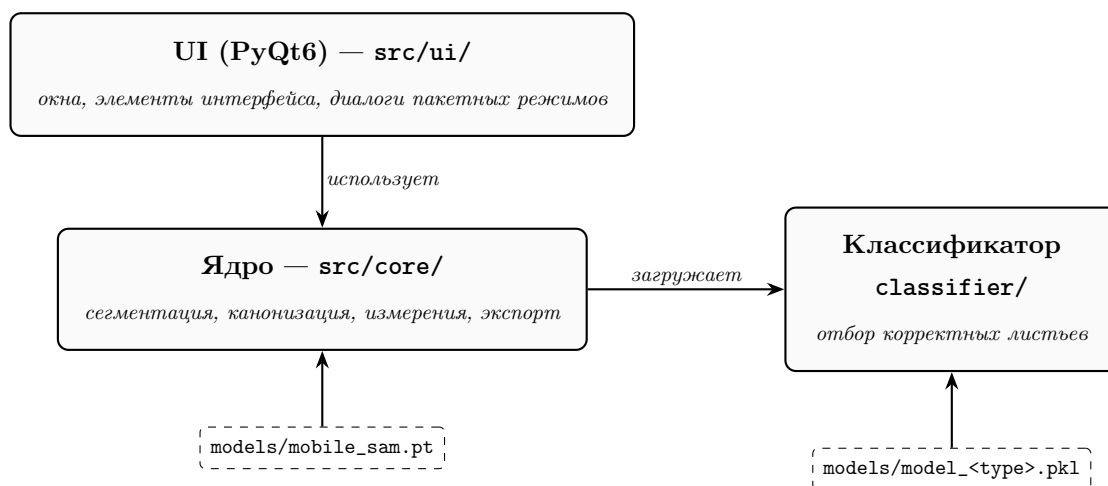


Рис. 1: Архитектура приложения: три независимых слоя и веса моделей, подгружаемые на старте

Точка входа — `src/main.py`: загружает веса MobileSAM, создаёт единственный экземпляр `Predictor` (синглтон, общий для всех режи-

мов) и открывает главное окно навигации (рис. 2). Из него доступны три режима: интерактивная сегментация (раздел 3.4), параметрические измерения по готовым фрагментам (раздел 3.6) и автоматическая сегментация и параметрические измерения (раздел 3.7).

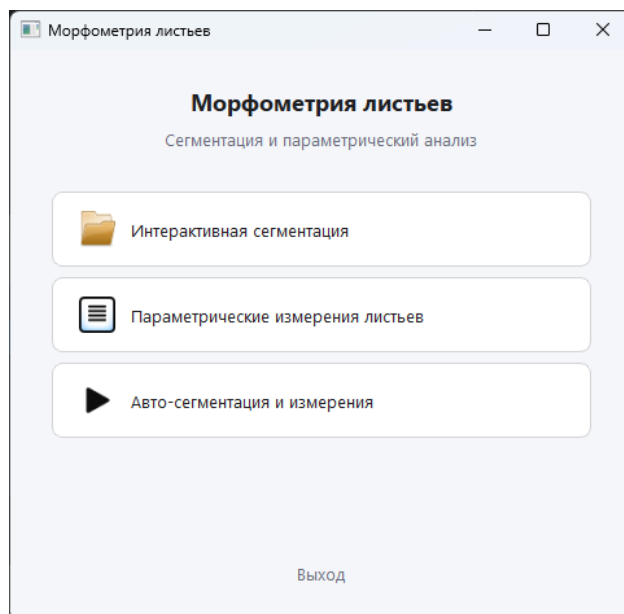


Рис. 2: Главное меню приложения

Оба пакетных режима реализованы поверх общего конвейера обработки с подключаемыми источниками фрагментов (см. раздел 3.6).

Конвенция имён файлов. Все три источника артефактов — ручная сегментация (раздел 3.4), автоматическая сегментация и параметрические измерения (раздел 3.7) и пользовательский ввод — порождают и потребляют файлы по единому шаблону: `{stem}_leaf_crop_{idx:04d}.png` для фрагмента, `{stem}_mask_{idx:04d}.png` для маски и `{crop_stem}_vis.png` для визуализации измерений. Реализация и Qt-фильтры сосредоточены в модуле `file_naming.py`, что позволяет смешивать в одной папке результаты разных источников и переиспользовать их между режимами без отдельной конвертации.

3.4. Интерактивная сегментация

Интерактивный режим (окно `SegmentationWindow`) реализует основной сценарий работы биолога: эксперт самостоятельно выбирает листья на изображении, а система строит маску по подсказке и при необходимости даёт инструменты ручной коррекции (рис. 3). Типовой рабочий цикл показан на рис. 5.

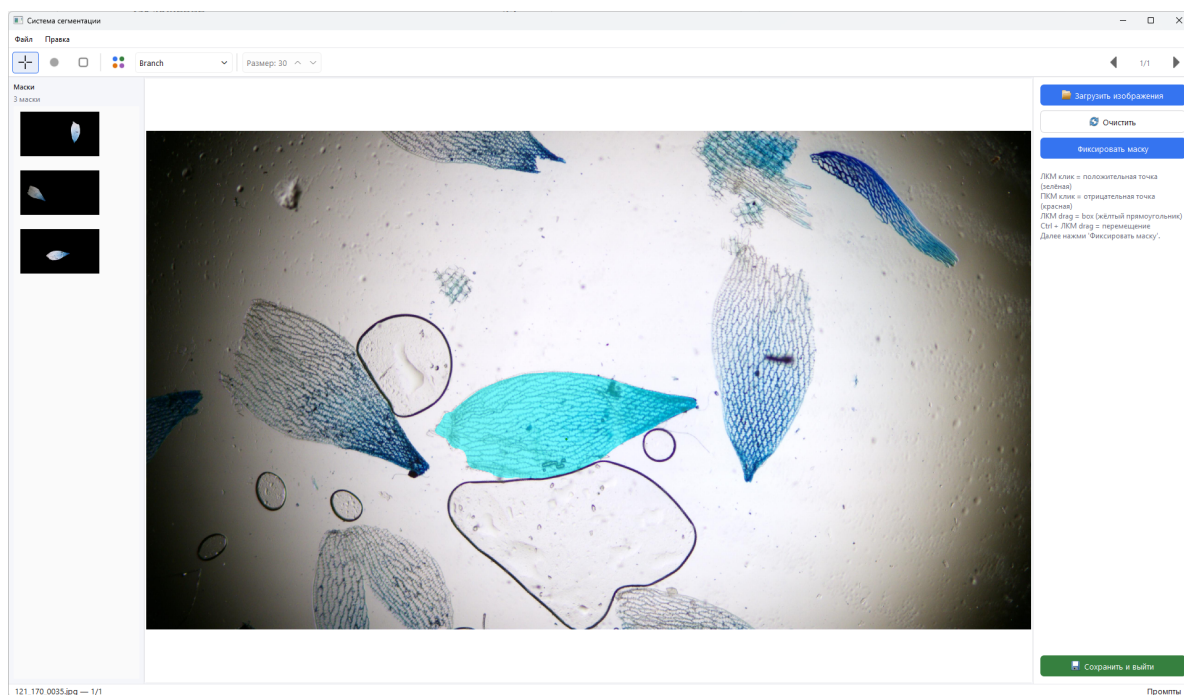


Рис. 3: Окно интерактивной сегментации

Загрузка серии и навигация. Окно работает с серией изображений. Загрузка набора файлов выполняется по нажатию кнопки «Загрузить» в боковой панели или через пункт меню «Файл → Загрузить изображения» (Ctrl+O): выбранные снимки добавляются в очередь обработки и проходятся последовательно. Перемещение между ними выполняется кнопками «вперёд» и «назад» в панели инструментов; для каждого изображения сохраняется собственный буфер масок, что позволяет накопить разметку по всей серии за один сеанс работы.

Холст и режимы работы. Центральный элемент интерфейса — `SegmentationCanvas` на основе `QGraphicsView` с поддержкой зума к курсору и панорамирования (Ctrl + ЛКМ). Холст имеет два режима.

В режиме *подсказок* клики левой и правой кнопкой мыши задают позитивные и негативные опорные точки, а перетаскивание ЛКМ — ограничивающий прямоугольник; после каждого изменения автоматически вызывается `Predictor.predict`, и над изображением отображается полупрозрачная маска. В режиме *редактирования* ЛКМ работает кистью, а ПКМ — ластиком; диаметр кисти регулируется ползунком в панели инструментов в диапазоне 1...200 пикселей. Маска хранится в виде ARGB-слоя (`QImage.Format_ARGB32`); переключение между режимами не теряет промежуточный результат.

Буфер масок. Подтверждённые маски попадают в список `MasksBuffer` (левая панель). Для каждой маски сохраняются два файла — собственно бинарная маска и RGB-фрагмент, прошедший канонизацию (раздел 3.5); при выходе из окна они копируются в выбранную пользователем директорию.

Кнопка авто-сегментации. В панели инструментов доступна кнопка автоматической сегментации текущего изображения для выбранного в панели типа листа. В буфер добавляются все найденные корректные листья (рис. 4) — эксперту остаётся при необходимости скорректировать ошибочные маски и доразметить пропущенные.

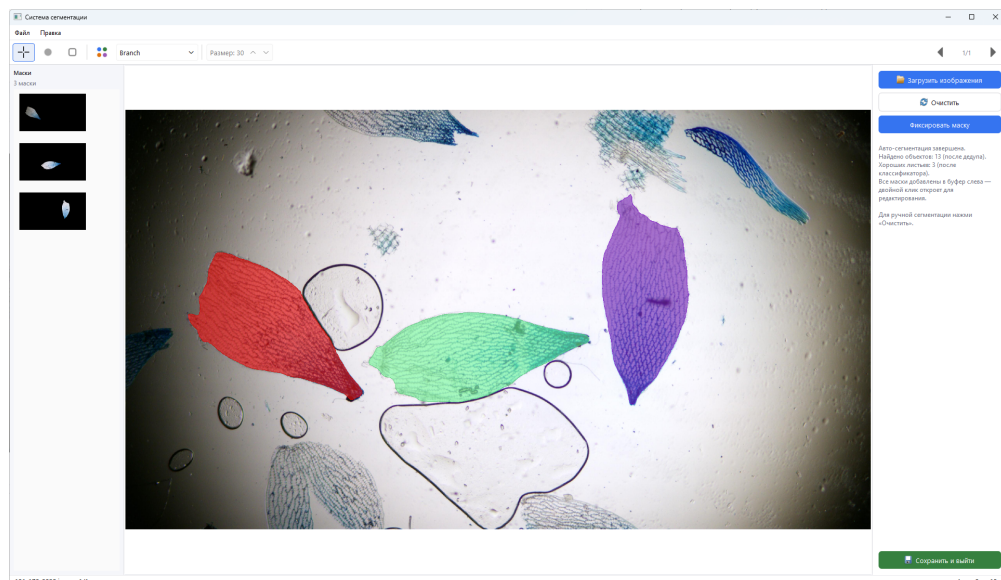


Рис. 4: Результат нажатия кнопки авто-сегментации: на холсте отображены только листья, прошедшие классификатор отбора

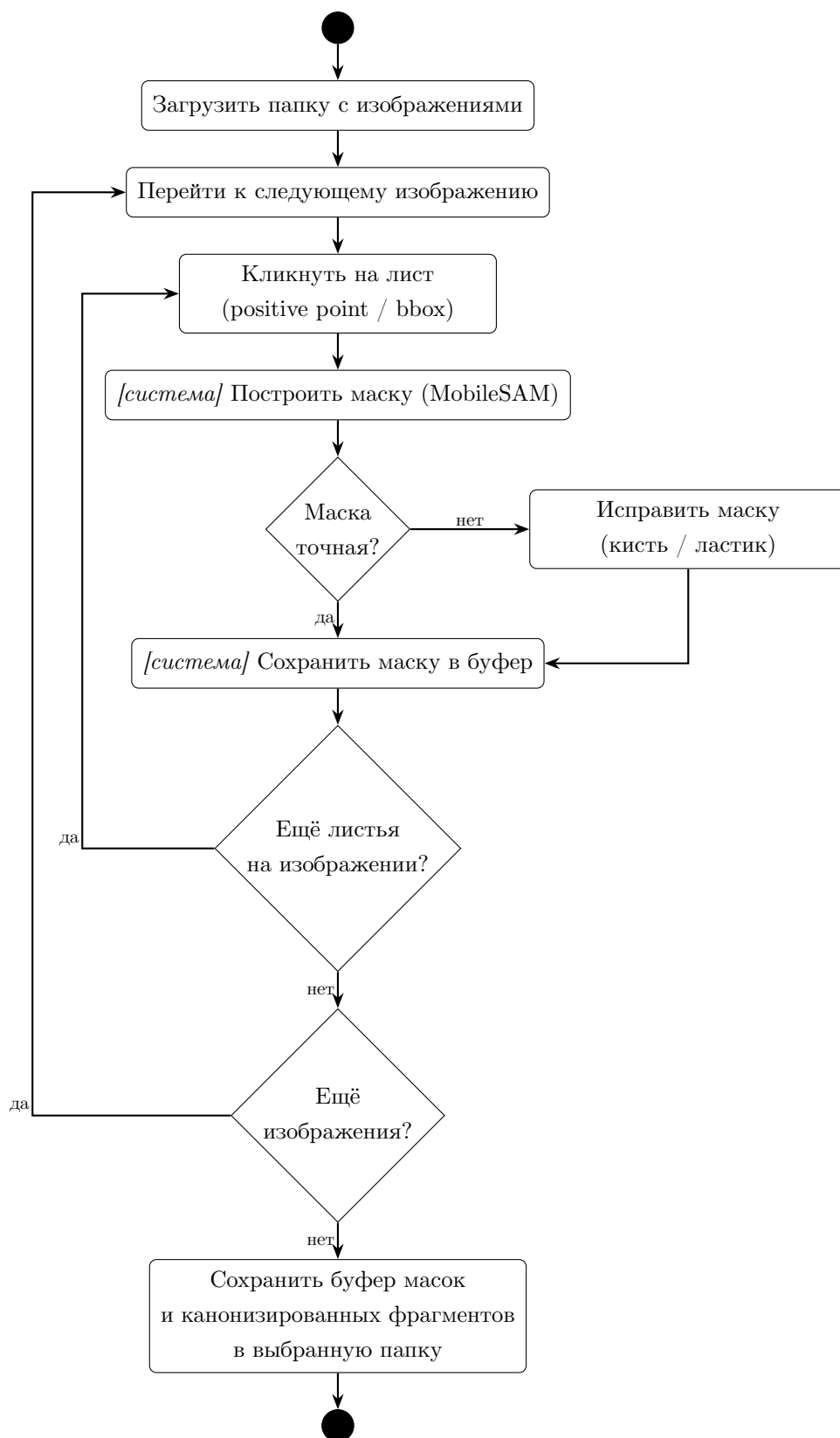


Рис. 5: Диаграмма активностей: рабочий цикл пользователя в интерактивном режиме

3.5. Канонизация листа

Маска, полученная сегментирующей моделью или ручной коррекцией, ориентирована произвольно и может содержать мелкие артефакты. Перед измерениями и классификацией её приводят к *каноническому* виду: главная ось листа вертикальна, апекс сверху, фон обнулён (рис. 6).

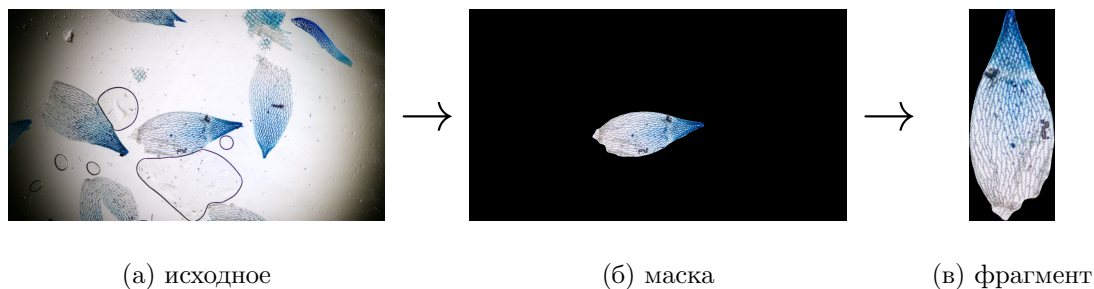


Рис. 6: Преобразование листа в конвейере: исходное изображение → очищенная маска → канонизированный фрагмент

Шаг 1: очистка маски. Берётся самый крупный внешний контур, его внутренняя область заполняется. Так удаляются мелкие посторонние компоненты и внутренние дыры.

Шаг 2: канонизация ориентации. Ориентация листа определяется методом главных компонент (Principal Component Analysis, PCA) [18]: по координатам активных пикселей маски строится матрица ковариации 2×2 и для неё вычисляются собственные векторы. Собственный вектор с наибольшим собственным значением даёт удлинённую (главную) ось листа. Изображение поворачивается аффинным преобразованием так, чтобы эта ось совпадала с вертикалью. После поворота определяется направление апекса: если в верхней половине ограничивающего прямоугольника больше активных пикселей, чем в нижней, изображение переворачивается — так апекс всегда оказывается сверху. Финальный шаг — обрезка по ограничивающему прямоугольнику активных пикселей.

3.6. Параметрические измерения листа

Алгоритм измерений

Модуль `leaf_measure.py` принимает на вход RGB-фрагмент в каноническом виде (раздел 3.5): главная ось вертикальна, апекс сверху, фон обнулён. Это позволяет отказаться от повторного построения контура и анализа главных компонент: все измерения сводятся к работе с маской активных пикселей в системе координат самого фрагмента (рис. 7).

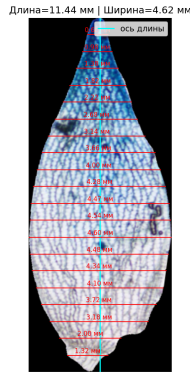


Рис. 7: Визуализация измерений одного листа: ось длины и поперечные сечения (для наглядности показано 20 сечений; по умолчанию используется семь)

Длина и максимальная ширина. Длина листа определяется как высота прямоугольника, ограничивающего активные пиксели фрагмента: $L = (y_{\max} - y_{\min} + 1) \cdot s$, где s — масштабный коэффициент (мм/пиксель), равный единице, если пользователь работает в пикселях. Максимальная ширина — ширина того же прямоугольника: $W = (x_{\max} - x_{\min} + 1) \cdot s$.

Поперечные сечения. Для каждой доли $f \in (0, 1)$ берётся строка $y = y_{\min} + \text{round}(f \cdot (y_{\max} - y_{\min}))$. В этой строке находятся крайний левый и крайний правый ненулевые пиксели x_l и x_r ; ширина в сечении равна $w_f = (x_r - x_l + 1) \cdot s$. По умолчанию используются семь сечений с долями 0,125; 0,25; 0,375; 0,5; 0,625; 0,75; 0,875, что соответствует равномерному разбиению длины листа на восемь интервалов.

Результат. Метод возвращает структуру `LeafMetrics` с полями `length`, `width` и словарём `widths`: `dict[float, SectionWidth]` — для

каждой доли хранятся сама ширина и координаты крайних точек.

Режим замеров по готовым фрагментам

Пакетный режим запускается из главного меню и используется, когда канонические фрагменты уже подготовлены ранее: сохранены интерактивным режимом сегментации (раздел 3.4), получены автоматическим режимом (раздел 3.7) или предоставлены пользователем напрямую. Параметры запуска задаются в диалоге настроек (рис. 8): набор файлов, выходная директория, число поперечных сечений, масштабный коэффициент, формат экспорта (по умолчанию — CSV).

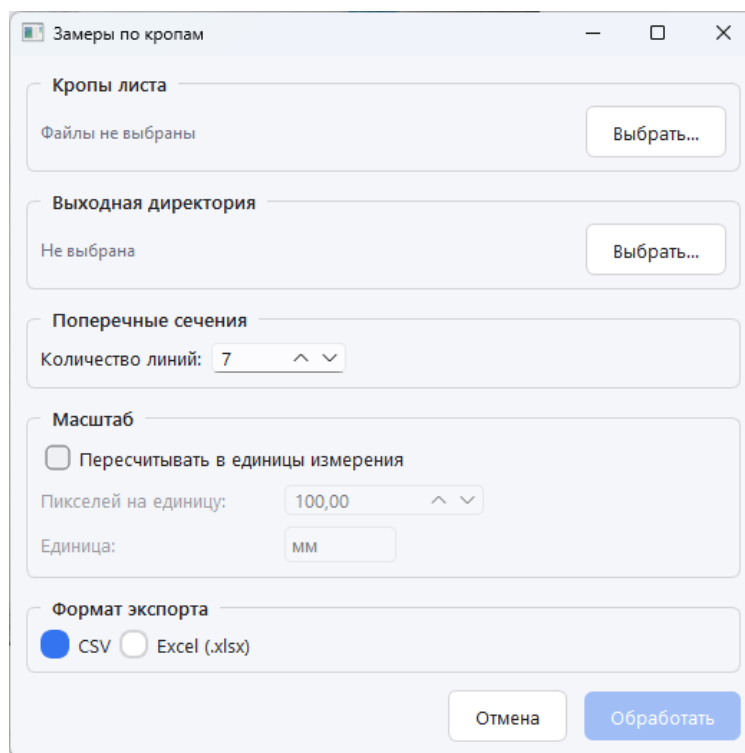


Рис. 8: Диалог настроек пакетной обработки

Каждый выбранный файл читается с диска по принятой конвенции имён (раздел 3.3) и подаётся на алгоритм измерений; результаты по всей серии накапливаются и в конце экспортируются в таблицу. По завершении пользователь получает диалог с числом обработанных файлов, числом ошибок и путём к итоговой таблице.

Помимо таблицы, для каждого обработанного фрагмента сохраня-

ется изображение `*_vis.png` с нанесённой осью длины и поперечными сечениями (см. рис. 7). В режиме экспорта в XLSX миниатюра этой визуализации встраивается в ячейку таблицы.

3.7. Автоматическая сегментация и параметрические измерения

В этом режиме система обрабатывает серию изображений без участия пользователя. Конвейер обработки одного снимка состоит из пяти шагов; на выходе всей серии — сводная таблица морфометрических измерений.

Шаг 1: генерация кандидатов. Сегментирующая модель запускается в автоматическом режиме (Automatic Mask Generator, AMG): без пользовательских подсказок выдаёт маски всех найденных объектов.

Шаг 2: дедупликация. AMG генерирует десятки масок-кандидатов; среди них часто встречаются дубликаты — несколько масок одного и того же листа разных размеров. Маски сортируются по убыванию площади и попарно отбрасываются по порогу $\text{IoU} > 0,9$.

Шаг 3: канонизация. К каждой выжившей маске применяется конвейер канонизации листа (раздел 3.5). На выходе — список пар (очищенная маска, канонический RGB-фрагмент).

Шаг 4: фильтрация классификатором отбора. Каждый канонический фрагмент маркируется обученным классификатором (раздел 4) одним из трёх классов; в итоговую таблицу попадают только фрагменты класса `good_leaf`.

Шаг 5: расчёт параметров и экспорт. К каждому отобранному фрагменту применяется алгоритм параметрических измерений (раздел 3.6); результаты по всей серии накапливаются и экспортируются в сводную таблицу. Помимо таблицы сохраняются маски и канонические RGB-фрагменты.

Режим открывается из главного меню; диалог настроек тот же, что у режима замеров, с одним дополнительным полем — выпадающим спис-

ком выбора типа листа, по которому подгружается соответствующая модель отбора.

3.8. Оптимизация производительности автоматической сегментации

Прямолинейная реализация обрабатывала один кадр 3840×2160 за $\approx 17,7$ с, что почти вдвое превышает предусмотренный требованиями (раздел 3.1) бюджет в 10 с на изображение. Последовательное применение семи оптимизаций снизило это время до $\approx 1,0$ с (рис. 9). Замеры выполнены на 4К-кадре с 13 масками.

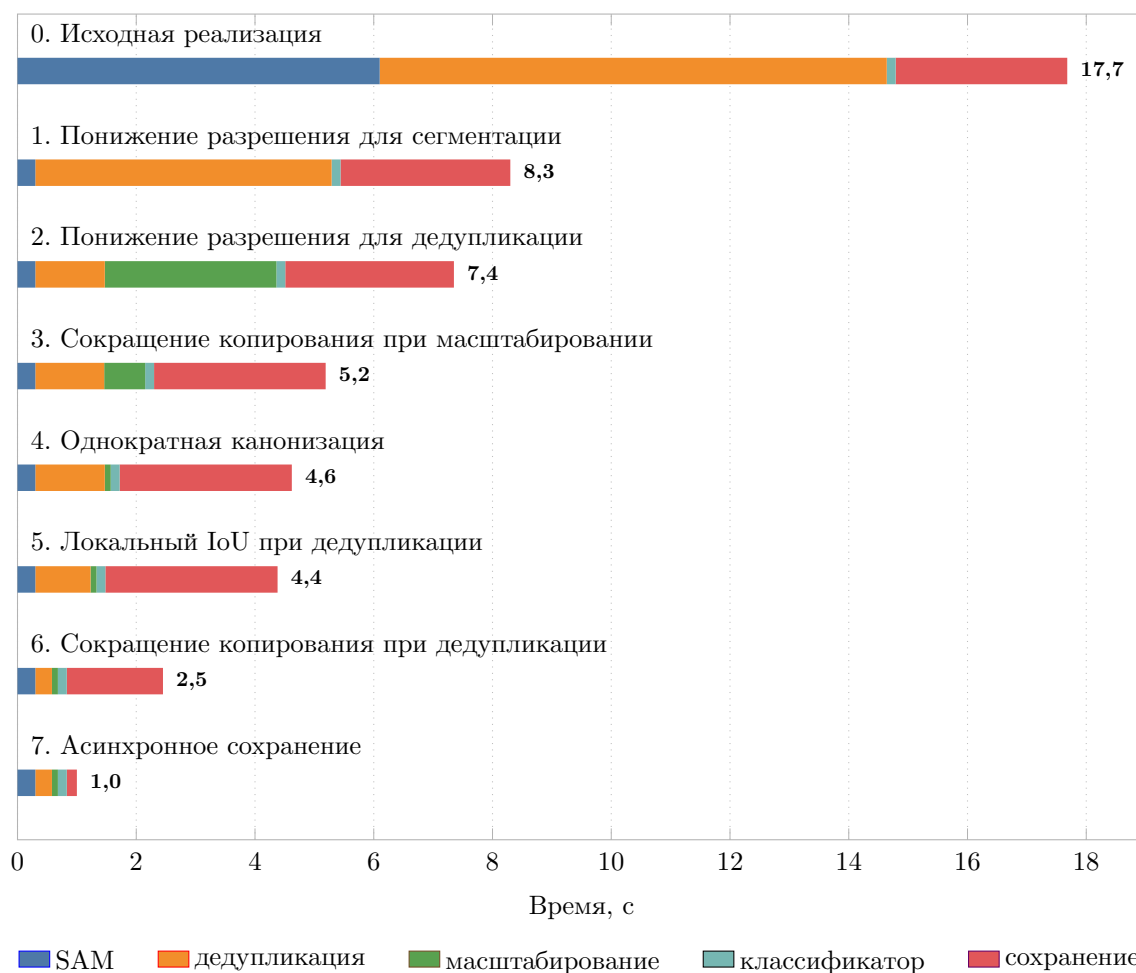


Рис. 9: Время одного 4К-кадра в конвейере автоматической сегментации по шагам оптимизации

Семь шагов из диаграммы группируются в пять приёмов.

Понижение входного разрешения (шаги 1–2). Кодировщик SAM принудительно приводит вход к квадрату 1024×1024 , поэтому стоимость самого инференса от исходного разрешения не зависит, но пост-обработка масок (немаксимальное подавление, фильтрация по качеству) масштабируется как $O(W \cdot H \cdot N)$. Подача на вход AMG вдвое уменьшенной копии кадра ускорила сегментацию приблизительно в 13 раз. Аналогично, дедупликация и канонизация выполняются на той же уменьшенной копии — до финального масштабирования выживших масок к исходному разрешению, — что сократило время дедупликации приблизительно в 4 раза.

Сокращение копирования кадра (шаги 3, 6). В двух циклах — финальном масштабировании и канонизации внутри дедупликации — для каждой маски выполнялась полная копия кадра (≈ 25 МБ для исходного разрешения и ≈ 6 МБ для уменьшенной копии), после чего пиксели вне маски обнулялись. Замена на копирование только области, ограниченной маской, устранила сотни мегабайт лишних данных и существенно ускорила аффинные преобразования.

Локальный IoU при дедупликации (шаг 5). Изначально попарный IoU между двумя масками считался в лоб: для каждой пары выполнялось два полных прохода по всем $H \times W$ пикселям обеих масок — отдельно для пересечения и объединения; при ~ 17 масках-кандидатах это давало десятки полнокадровых обходов. Замена устроена в два шага. Во-первых, перед попарным сравнением проверяется пересечение ограничивающих прямоугольников масок: если прямоугольники не пересекаются, маски заведомо не пересекаются и $\text{IoU} = 0$ — пара отсеивается без единого пиксельного сравнения. Во-вторых, для пересекающихся прямоугольников $|A \cap B|$ считается только в их общей области, а объединение — по формуле $|A \cup B| = |A| + |B| - |A \cap B|$ из заранее посчитанных площадей и только что найденного пересечения, без второго полного прохода.

Однократная канонизация (шаг 4). Дедупликация уже выполняла канонизацию каждого выжившего листа на уменьшенной копии кадра, а финальный цикл масштабирования игнорировал этот результат и

канонизировал тот же лист повторно на полном разрешении. Замена на билинейное масштабирование уже канонизированного фрагмента к нужному размеру избавила от двойной работы. Возникающее размытие в 2–3 пикселя не критично.

Асинхронное сохранение (шаг 7). Запись масок на диск синхронно с интерфейсом давала $\approx 2,9$ с зависания. Запись вынесена в фоновый пул потоков; для отрисовки миниатюр 128×128 один раз на изображение строится уменьшенная копия исходника (≈ 250 КБ вместо 25 МБ), из которой и берутся все иконки. Интерфейс больше не блокируется на сохранение, а синхронизация с фоновыми записями выполняется только там, где она действительно нужна: при сохранении результатов и выходе из окна ждём завершения всех записей перед копированием файлов в выходную папку, а при редактировании или удалении уже сохранённой маски ждём её конкретной записи, чтобы перечитать файл с диска.

Итог. Время обработки одного 4К-кадра сократилось с 17,7 до 1,0 с — примерно в восемнадцать раз. Кнопка авто-сегментации в окне интерактивной разметки (раздел 3.4) теперь отрабатывает за секунду: задержка приемлема для интерактивного использования.

3.9. Тестирование

Все тесты исполняются автоматически при каждом пуше через GitHub Actions.

Модульные тесты (≈ 160 случаев) покрывают чистые функции ядра: конвенцию имён артефактов, чтение изображений, препроцессинг маски и PCA-канонизацию, API измерителя, экспорт CSV/XLSX, конвейер обработки и адаптеры классификатора.

Контроль точности измерений выполняется на контрольной выборке из 9 снимков, для которых эксперт-биолог вручную замерил длину листа и семь поперечных ширин в пикселях. Тест загружает изображение и заранее подготовленную маску, прогоняет `transform_leaf` и `measure_leaf`, сверяет результат с эталоном по критерию $\max(\delta_{\text{отн}}, \delta_{\text{абс}})$:

для длины — 3 %, для сечений — 5 %.

Контроль точности сегментации и классификатора отбора выполняется на снимках с эталонными масками, размеченными вручную для каждого хорошего листа на изображении. Тест прогоняет полную цепочку автоматической сегментации с фильтром классификатора и сопоставляет полученные `good_leaf`-маски с эталонными по IoU; пара засчитывается как совпадение только при $\text{IoU} \geq 0,95$, поэтому грубая по форме маска одновременно идёт в зачёт как промах сегментации. Подсчитываются ложные срабатывания (лишние `good_leaf`-маски) и ложные пропуски (эталонные листья, не попавшие в выдачу — либо SAM пропустил, либо классификатор отверг, либо маска слишком груба).

3.10. Сборка и публикация

Целевая аудитория приложения — сотрудники биологической лаборатории, для которых установка Python и его зависимостей не является штатной операцией. Доставка реализована в виде автономной сборки под Windows 64 на базе PyInstaller 6.11.1 в режиме `--onedir`, включающей интерпретатор Python, все библиотеки и веса моделей. Сборка использует только CPU и не требует ни предустановленного Python, ни системных зависимостей, ни GPU.

Итоговая сборка занимает ≈ 920 МБ и распространяется в виде RAR-архива (≈ 327 МБ); готовые сборки опубликованы в разделе [Releases](#) репозитория проекта. Запуск — двойным кликом по `sphagnum.exe`; стандартные потоки перенаправляются в файлы `stdout.log` и `stderr.log` в каталоге сборки.

4. Обучение классификатора отбора листьев

4.1. Постановка задачи

После сегментации (раздел 3.7) получается набор масок-кандидатов всех объектов на изображении: и корректные одиночные листья, пригодные для измерений, и брак — повреждённые, перекрывающиеся, нелистовые артефакты. Для отбора пригодных листьев обучается отдельный классификатор. Вход модели — канонический RGB-фрагмент (раздел 3.5). Выход — метка одного из трёх классов:

- **good_leaf** — корректный одиночный лист, пригодный для морфометрических измерений;
- **bad_leaf** — лист с дефектами (перекрытия с другими объектами, неполная форма, рваные края);
- **non_leaf** — объект нелистовой природы (фон, тень, артефакт сегментации).

Стоимость ошибок классификатора несимметрична: пропущенный лист безвозвратно теряется, тогда как ложно положительный фрагмент попадает в итоговую таблицу измерений и удаляется вручную. Поэтому в качестве рабочей точки фиксируется уровень $\text{recall} \geq 0,95$ на позитивном классе **good_leaf** — гарантия, что не более $\sim 5\%$ корректных листьев теряется на этапе фильтрации. Соответствующая **precision** при этом **recall** используется как основная метрика качества по всей главе.

4.2. Архитектура классификатора

Малый объём ручной разметки ($\sim 100 - 600$ позитивных примеров на тип листа, см. табл. 3) исключает дообучение свёрточной сети целиком — риск переобучения слишком высок. Поэтому классификатор реализован как двухэтапная модель: замороженный экстрактор признаков на основе свёрточной сети, предобученной на ImageNet, и градиентный бустинг XGBoost, обучаемый на этих признаках (рис. 10).

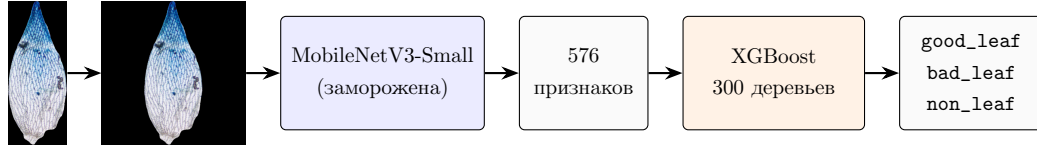


Рис. 10: Конвейер классификатора отбора масок

Извлечение признаков

Используется MobileNetV3-Small из `torchvision`, предобученная на ImageNet. Финальный классификационный слой заменяется на `Identity` (сеть возвращает 576-мерный эмбединг вместо 1000 ImageNet-логитов), все параметры заморожены. Каждый фрагмент вписывается в квадрат 512×512 пикселей с сохранением пропорций, нормализуется по статистикам ImageNet и пропускается через сеть; на выходе — вектор 576 признаков.

Параметры XGBoost

XGBoost (`XGBClassifier, multi:softprob`) с гиперпараметрами: 300 деревьев, максимальная глубина 6, темп обучения 0,1, `subsample = 0,8`, `colsample_bytree = 0,8`, `tree_method = "hist"`. Веса классов компенсируют дисбаланс (~ 100 – 600 позитивных против 1500 `bad_leaf` и ~ 1000 `non_leaf`) и задаются формулой:

$$w_c = \frac{N_{\text{total}}}{n_{\text{classes}} \cdot N_c}, \quad (1)$$

где N_c — число примеров класса c , N_{total} — общий размер обучающей выборки. Обоснование выбора численных значений приведено в разделе 4.5.

4.3. Сбор обучающей выборки

Размеченная выборка собрана автором работы с использованием интерактивного режима приложения (раздел 3.4): на каждом исходном снимке запускалась автосегментация, полученные кандидаты вручную

распределялись по трём классам.

Морфология листьев у разных групп *Sphagnum* существенно различается, поэтому позитивный класс `good_leaf_<type>` собран отдельно для каждого из трёх типов, а отказные классы `bad_leaf` (повреждённые или перекрытые листья) и `non_leaf` (артефакты сегментации, фон, тени) — общие для всех типов и не требуют повторной разметки. Полный датасет опубликован: <https://cloud.mail.ru/public/jvB9/ReNV4TrpS>.

Таблица 3: Распределение классов в обучающей выборке (по типам листа)

Тип листа	good_leaf	bad_leaf	non_leaf	Доля good
branch	566	1500	994	18,5 %
capillifolium	121	1500	994	4,6 %
girgensohnii	309	1500	994	11,0 %

Аугментация

Каждый фрагмент размножается в шесть детерминированных вариантов: горизонтальное отражение (два состояния), перемноженное на три цветовые трансформации — оригинал, осветление и затемнение по яркости, контрасту и насыщенности. Вертикальное отражение нарушило бы инвариант “апекс сверху” и не используется. Признаки MobileNetV3 вычисляются для каждого варианта один раз и кэшируются.

4.4. Результаты

Качество оценивается 5-кратной стратифицированной кросс-валидацией; на обучающих разбиениях применяется шестикратная аугментация, тестовое разбиение проходит без аугментации. По накопленным предсказаниям пяти разбиений в бинарной задаче `good_leaf` против объединения `bad_leaf` и `non_leaf` для каждого типа листа найден минимальный порог p_{good} , при котором $\text{recall} \geq 0,95$ (рис. 11).

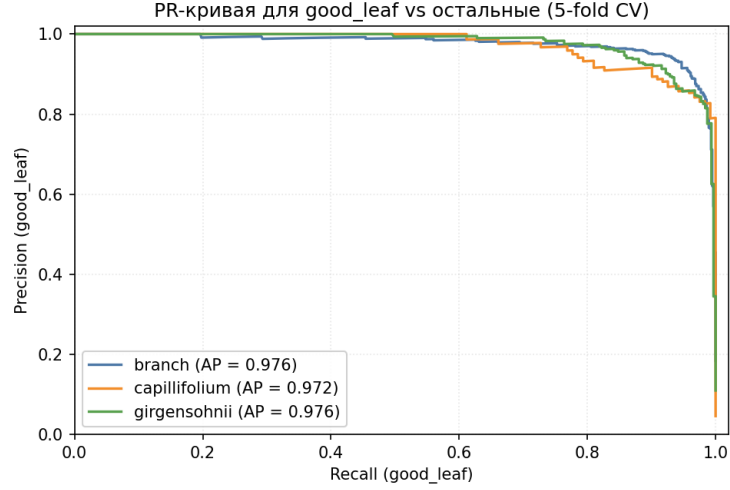


Рис. 11: PR-кривая классификатора для бинарной задачи `good_leaf` против остальных классов (5-кратная кросс-валидация)

Таблица 4: Метрики классификатора на рабочей точке $\text{recall} \geq 0,95$ (5-кратная кросс-валидация)

Тип листа	p_{good}	Precision	Recall	F1	AP
branch	0,22	$0,921 \pm 0,027$	0,95	0,935	$0,977 \pm 0,009$
capillifolium	0,24	$0,884 \pm 0,050$	0,95	0,916	$0,972 \pm 0,014$
girgensohnii	0,12	$0,882 \pm 0,049$	0,95	0,915	$0,978 \pm 0,009$

Precision и AP даны как среднее по 5 фолдам $\pm$ стандартное отклонение между фолдами. На всех трёх типах достигнут целевой recall при precision 0,88–0,92. Площадь под PR-кривой $> 0,97$ для всех типов означает, что модель хорошо разделяет позитивный класс почти на всём диапазоне порогов. Минимальная precision на `capillifolium` и `girgensohnii` связана с меньшим объёмом размеченных позитивных примеров (121 и 309 против 566 для `branch`).

Разброс по фолдам ($\pm 2,7$ п.п. на `branch`, ± 5 п.п. на `capillifolium` и `girgensohnii`) при усреднении по 5 разбиениям даёт стандартную ошибку среднего $\sigma_{\text{CV}} \approx 1\text{--}2$ п.п. — этим определяется уровень шума при сравнении архитектурных вариантов в разделе 4.5.

4.5. Сравнение с альтернативными архитектурами

Каждое архитектурное решение проверено отдельным экспериментом по тому же протоколу 5-кратной кросс-валидации, что и в табл. 4; метрика — precision при recall = 0,95 на классе `good_leaf`. Прикладные ограничения, принимаемые во внимание при выборе архитектуры: вычисления на CPU рабочей станции лаборатории, малый объём ручной разметки ($\sim 100 - 600$ позитивных примеров на тип листа), повторное обучение силами лаборатории при добавлении нового типа.

Выбор сети-экстрактора признаков

Сравнивались восемь замороженных экстракторов признаков с фиксированным итоговым классификатором XGBoost (табл. 5).

Таблица 5: Precision при recall = 0,95 для разных экстракторов признаков

Экстрактор	branch	capilli.	girgens.	CPU, мс	Вес, МБ
MobileNetV3-Small [25] (выбран)	0,913 $\pm$ 0,049	0,896 $\pm$ 0,039	0,889 $\pm$ 0,030	6	9,7
EfficientNet-B0 [27]	0,901 $\pm$ 0,038	0,826 $\pm$ 0,048	0,904 $\pm$ 0,027	13	20,5
DINOv2-small [8]	0,882 $\pm$ 0,035	0,896 $\pm$ 0,109	0,911 $\pm$ 0,029	192	84,2
MobileNetV3-Large [25]	0,874 $\pm$ 0,006	0,841 $\pm$ 0,062	0,877 $\pm$ 0,044	16	21,0
ResNet-50 [10]	0,896 $\pm$ 0,015	0,875 $\pm$ 0,108	0,889 $\pm$ 0,013	25	97,8
ConvNeXt-Tiny [7]	0,879 $\pm$ 0,044	0,857 $\pm$ 0,034	0,898 $\pm$ 0,020	25	109,1
ResNet-18 [10]	0,872 $\pm$ 0,027	0,740 $\pm$ 0,142	0,886 $\pm$ 0,046	11	44,7
MobileSAM-encoder [12]	0,772 $\pm$ 0,069	0,686 $\pm$ 0,081	0,801 $\pm$ 0,036	347	38,8

MobileNetV3-Small лидирует на **branch** (самой крупной выборке) и **capillifolium**, на **girgensohnii** уступает DINOv2-small на 2,2 п.п. — в пределах CV-шума σ_{CV} . DINOv2 проигрывает в 32 раза по скорости и в 8,7 раза по объёму весов — неприемлемо для рабочей станции без GPU. Кодировщик MobileSAM (переиспользование весов сегментатора) даёт просадку 8,8 – 21,0 п.п. при 58-кратном замедлении — разрывкратно превышает шум, эта замена однозначно отвергается.

Сравнение глубоких и ручных признаков

Морфометрическая базовая модель на 84 ручных признаках (форма и площадь, профиль ширины, эллиптические Фурье-дескрипторы [19],

моменты X_y [14], симметрия, кривизна у апекса) с тем же XGBoost сравнена с признаками MobileNet (табл. 6).

Таблица 6: Precision при recall = 0,95: глубокие и ручные признаки

Признаки	branch	capilli.	girgens.
MobileNetV3-Small (576-d, выбран)	0,913 ± 0,049	0,896 ± 0,039	0,889 ± 0,030
84 ручных	0,793 ± 0,074	0,577 ± 0,159	0,616 ± 0,052

Разрыв 12,0–31,9 п.п. означает, что контурная геометрия не описывает текстурный сигнал, существенный для отбраковки повреждённых и нелистовых объектов.

Выбор классификатора

На зафиксированных признаках MobileNet сравнивались шесть альтернатив XGBoost (табл. 7).

Таблица 7: Precision при recall = 0,95 для разных классификаторов по верх признаков MobileNetV3-Small

Модель	branch	capilli.	girgens.
XGBoost (выбран)	0,921 ± 0,027	0,884 ± 0,050	0,882 ± 0,049
RBF-SVM	0,888 ± 0,039	0,906 ± 0,052	0,918 ± 0,036
MLP	0,892 ± 0,037	0,911 ± 0,059	0,897 ± 0,025
LogReg	0,860 ± 0,050	0,876 ± 0,093	0,845 ± 0,039
RandomForest [4]	0,862 ± 0,035	0,717 ± 0,096	0,873 ± 0,049
LinearSVM	0,814 ± 0,015	0,822 ± 0,064	0,877 ± 0,034
kNN	0,801 ± 0,056	0,749 ± 0,092	0,821 ± 0,052

XGBoost — лидер на **branch** (самой крупной выборке); на **capillifolium** и **girgensohnii** он уступает MLP и RBF-SVM на 2,7–3,6 п.п. — порядка $1-2\sigma_{CV}$, на грани шума и без устойчивого преимущества альтернатив. XGBoost оставлен как лидер на самой крупной выборке плюс встроенная поддержка **sample_weight** для компенсации дисбаланса. Линейные модели (LogReg, LinearSVM) и kNN отстают на 5–10 п.п. — разрывкратно превышает шум.

Параметры XGBoost

Для проверки выбранных значений и оценки, нужна ли отдельная подстройка под каждый тип листа, выполнен поиск по сетке ($n_estimators \in [50; 1500]$, $max_depth \in [2; 12]$, $learning_rate \in [0,01; 0,3]$, $subsample$, $colsample_bytree \in [0,5; 1,0]$): каждый параметр перебирался независимо при остальных, зафиксированных на выбранных значениях.

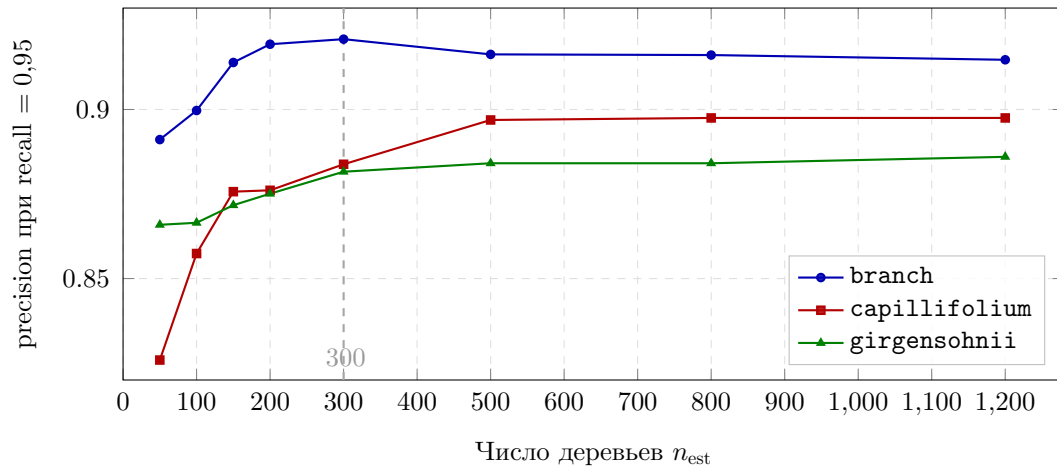


Рис. 12: Зависимость precision (при recall = 0,95) от числа деревьев XGBoost при остальных параметрах, фиксированных на выбранных значениях. Пунктир — выбранное $n_{est} = 300$.

На **branch** (самой крупной выборке) выбранные значения уже стоят в максимуме сетки по всем пяти параметрам — одиночное изменение любого из них только ухудшает precision. На меньших выборках **capillifolium** и **girgensohnii** лучший прирост от изменения одного параметра не превышает 2 п.п., что не выходит за шум σ_{CV} . Лучшие значения по разным типам листа взаимно противоречат (например, max_depth : 6 / 4 / 5; $subsample$: 0,8 / 0,7 / 1,0) — характерное проявление переобучения на конкретные разбиения, а не сигнал о наличии предпочтительной конфигурации под отдельный тип. Поэтому единая конфигурация ($n_estimators = 300$, $max_depth = 6$, $learning_rate = 0,1$, $subsample = 0,8$, $colsample_bytree = 0,8$) оставлена для всех трёх моделей.

Число классов

Бинарная постановка (good против объединения bad и non) проверена на той же выборке и том же XGBoost (табл. 8).

Таблица 8: Precision при recall = 0,95: трёхклассовая и бинарная постановка

Постановка	branch	capilli.	girgens.
3 класса (выбрана)	0,921 ± 0,027	0,884 ± 0,050	0,882 ± 0,049
2 класса (good против остального)	0,895 ± 0,042	0,839 ± 0,051	0,896 ± 0,044

3-классовая постановка выигрывает на **branch** (+2,6 п.п.) и **capillifolium** (+4,5 п.п., $\sim 2\sigma_{CV}$), уступает на **girgensohnii** (−1,5 п.п., в пределах σ_{CV}). На самом малом позитивном классе (**capillifolium**, 121 образец) разрыв максимальный — различение типов отказа даёт дополнительный обучающий сигнал, особенно ценный при малом числе положительных примеров.

Итог раздела

Главный ограничитель качества классификатора — объём и качество размеченной выборки (**capillifolium** — 121 позитивный пример, **girgensohnii** — 309, табл. 3), а не архитектурные решения: расширение и уточнение разметки сдвинут рабочую точку существеннее любой замены сети-экстрактора, классификатора или подстройки гиперпараметров.

5. Эксперимент и апробация

5.1. Цель и методология

Цель эксперимента — количественно оценить точность автоматических измерений системы сравнением с ручными измерениями специалиста лаборатории Гербарий ГБС РАН. Тестовая выборка — 9 изображений листьев *Sphagnum* трёх разных типов; для каждого образца специалист вручную замерил длину листа по главной оси и ширину в семи равноотстоящих сечениях (на долях 0,125; 0,25; 0,375; 0,5; 0,625; 0,75; 0,875 от полной длины). Те же изображения обработаны системой в автоматическом режиме сегментации и параметрических измерений.

5.2. Сравнение длин листьев

Таблица 9: Сравнение автоматических и ручных измерений длины листа (пиксели)

№	Авто	Вручную	Δ , px	APR, %
1	1131,9	1106,2	+25,7	2,3
2	734,2	703,2	+31,0	4,4
3	1142,9	1132,0	+10,9	1,0
4	1511,2	1446,2	+65,0	4,5
5	2018,4	1992,2	+26,2	1,3
6	850,9	766,9	+84,0	11,0
7	1899,2	1680,4	+218,8	13,0
8	855,2	829,2	+26,0	3,1
9	443,8	408,2	+35,6	8,7
Итого			+58,1	5,5

Средняя абсолютная погрешность составила $MAE = 58,1$ px при $RMSE = 84,0$ px и средней относительной погрешности $MAPE = 5,5$ %. Коэффициент корреляции $R = 0,993$ свидетельствует о высокой линейной согласованности автоматических и ручных измерений.

Наибольшие отклонения наблюдаются у образцов № 7 и № 6 ($APR = 13,0\%$ и $11,0\%$) и объясняются субъективностью ручного измерения: у этих образцов в основании листа имеются выросты — структуры, относящиеся к листу, которые эксперт при ручной разметке решил не включать в измеряемую длину. Алгоритм же по полной маске учитывает их как часть листа, что и даёт расхождение. Для остальных образцов с более регулярной формой основания относительная погрешность длины не превышает 5% .

5.3. Сравнение поперечных профилей ширины

В таблице 10 представлены метрики погрешности для поперечных профилей по каждому из девяти образцов. Анализ проводился по всем 63 значениям (9 листьев $\times$ 7 сечений).

Таблица 10: Метрики погрешности поперечных профилей ширины (пиксели)

№	Bias	MAE	RMSE	MAPE, %
1	−3,9	7,7	10,7	1,95
2	−3,9	9,2	11,2	1,75
3	−1,2	2,3	2,7	0,60
4	−12,9	13,7	16,3	1,54
5	−14,3	14,4	20,8	2,41
6	+7,1	13,7	16,2	3,97
7	+12,1	20,7	25,3	2,44
8	−1,8	3,9	4,2	0,96
9	−3,5	4,4	7,1	1,43
Итого	−2,5	10,0	14,6	1,90

Суммарная MAPE по поперечным профилям составила **1,90 %**. Коэффициент корреляции $R = 0,998$ при 63 парах наблюдений подтверждает высокую согласованность алгоритма сканирования поперечных сечений с ручными измерениями.

5.4. Итоговые показатели

В таблице 11 сведены итоговые метрики по двум типам измерений.

Таблица 11: Итоговые метрики точности автоматических измерений (Bias, MAE, RMSE — в пикселях)

Тип измерения	N	Bias	MAE	RMSE	MAPE, %	R
Длина листа	9	+58,1	58,1	84,0	5,5	0,993
Поперечные профили ширины	63	-2,5	10,0	14,6	1,90	0,998

Измерение поперечных профилей демонстрирует высокую точность — погрешность менее 2 % приемлема для сравнительного морфометрического анализа. Длина листа имеет систематическое завышение порядка 5–6 % относительно ручных измерений: алгоритм по полной маске учитывает выросты у основания листа, которые эксперт при ручной разметке субъективно исключает (см. анализ образцов № 7 и № 6 выше).

По итогам апробации была признана корректность результатов алгоритма на тестовой выборке. Расхождение в длине отнесено к субъективности ручной разметки, а не к ошибке алгоритма.

5.5. Сравнение по затратам времени

Среднее время ручного измерения одного листа экспертом составило $2,6 \pm 0,4$ минут; автоматическая обработка одного изображения заняла $\approx 1,0$ с (раздел 3.8), что соответствует ускорению приблизительно на два порядка.

5.6. Апробация

Апробация системы проведена консультантом работы — сотрудником лаборатории Гербарий ГБС РАН А. В. Шкурко. Тестирование выполнено на рабочих станциях под управлением Windows 10 x64 и Windows 11 x64; основная конфигурация — процессор Intel Core i7-13620H (2,40 ГГц), 16 ГБ оперативной памяти, без выделенного графического ускорителя.

Установка и запуск приложения прошли без затруднений на обеих версиях операционной системы. Интерфейс охарактеризован как интуитивно понятный, отклик на действия пользователя — мгновенный.

На ветвевых листьях консультантом отмечена возможность полностью автоматической обработки без ручной коррекции масок.

Отдельно отмечено, что система корректно построила маски и выполнила измерения для листьев с бахромчатыми (рваными) краями, несмотря на то что классификатор отбора обучался на образцах с замкнутым гладким контуром.

Из замечаний отмечено одно: при включённой тёмной теме Windows текст всплывающих подсказок сливался с фоном панели. Замечание устранено: для приложения зафиксированы светлая палитра и стиль отрисовки виджетов Fusion, исключаяющие влияние системной темы.

По итогам апробации консультант высоко оценил функциональные возможности системы; апробация признана успешной.

Заключение

В ходе работы разработана система для автоматической сегментации и морфометрических измерений листьев *Sphagnum* в виде настольного приложения под Windows. Были выполнены следующие задачи:

1. Сформулированы требования к разрабатываемой системе.
2. Спроектирована архитектура и реализовано приложение с тремя режимами работы: интерактивная сегментация, параметрические измерения по готовым фрагментам и автоматическая сегментация с измерениями. Время обработки одного 4К-кадра в автоматическом режиме — $\approx 1,0$ с.
3. Собрана и размечена обучающая выборка для классификатора отбора листьев — около 3500 канонизированных фрагментов, в том числе 996 корректных листьев трёх типов *Sphagnum*.
4. Обучен классификатор отбора для трёх типов листьев *Sphagnum*; на классе `good_leaf` при гарантированном $\text{recall} = 0,95$ precision составляет $0,88-0,92$, $\text{AP} > 0,97$.
5. Проведено сравнение автоматических измерений с ручной разметкой эксперта на 9 контрольных снимках: МАРЕ длины = 5,5 %, МАРЕ поперечных профилей = 1,90 %. Апробация подтвердила работоспособность и точность системы.

Артефакты работы:

- Исходный код приложения:
https://github.com/pavelrubanov/segmentation_system
- Размеченный датасет:
<https://cloud.mail.ru/public/jvB9/ReNV4TrpS>
- Готовая сборка под Windows 64:
https://github.com/pavelrubanov/segmentation_system/releases

Список литературы

- [1] Array programming with NumPy / Charles R. Harris, K. Jarrod Millman, Stéfan J. van der Walt et al. // [Nature](#). — 2020. — Vol. 585. — P. 357–362.
- [2] Bookstein Fred L. Morphometric Tools for Landmark Data: Geometry and Biology. — Cambridge : Cambridge University Press, 1991.
- [3] Bradski Gary. The OpenCV Library // Dr. Dobb's Journal of Software Tools. — 2000. — Vol. 25, no. 11. — P. 120–126.
- [4] Breiman Leo. Random Forests // [Machine Learning](#). — 2001. — Vol. 45, no. 1. — P. 5–32.
- [5] Chen Tianqi, Guestrin Carlos. [XGBoost: A Scalable Tree Boosting System](#) // Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD '16). — New York, NY, USA : ACM, 2016. — P. 785–794.
- [6] Clymo R. S., Hayward P. M. [The Ecology of Sphagnum](#) // Bryophyte Ecology / Ed. by A. J. E. Smith. — Dordrecht : Springer, 1982. — P. 229–289.
- [7] [A ConvNet for the 2020s](#) / Zhuang Liu, Hanzi Mao, Chao-Yuan Wu et al. // Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR). — 2022. — P. 11976–11986.
- [8] DINOv2: Learning Robust Visual Features without Supervision / Maxime Oquab, Timothée Darcet, Théo Moutakanni et al. // Transactions on Machine Learning Research (TMLR). — 2024. — [2304.07193](#).
- [9] Daniels R. E., Eddy A. Handbook of European Sphagna. — Abbots Ripton, Huntingdon : Natural Environment Research Council, Institute of Terrestrial Ecology, 1985. — ISBN: [0-904282-82-1](#).
- [10] [Deep Residual Learning for Image Recognition](#) / Kaiming He, Xiangyu Zhang, Shaoqing Ren, Jian Sun // Proceedings of the IEEE

Conference on Computer Vision and Pattern Recognition (CVPR). — 2016. — P. 770–778.

- [11] Dryden Ian L., Mardia Kanti V. Statistical Shape Analysis. — New York : John Wiley & Sons, 1998.
- [12] Zhang Chaoning, Han Dongshen, Qiao Yu et al. Faster Segment Anything: Towards Lightweight SAM for Mobile Applications. — 2023. — [2306.14289](#).
- [13] Fiji: an open-source platform for biological-image analysis / Johannes Schindelin, Ignacio Arganda-Carreras, Erwin Frise et al. // [Nature Methods](#). — 2012. — Vol. 9, no. 7. — P. 676–682.
- [14] Hu Ming-Kuei. Visual Pattern Recognition by Moment Invariants // [IRE Transactions on Information Theory](#). — 1962. — Vol. 8, no. 2. — P. 179–187.
- [15] Hunter John D. Matplotlib: A 2D Graphics Environment // [Computing in Science & Engineering](#). — 2007. — Vol. 9, no. 3. — P. 90–95.
- [16] ImageNet Large Scale Visual Recognition Challenge / Olga Russakovsky, Jia Deng, Hao Su et al. // [International Journal of Computer Vision](#). — 2015. — Vol. 115, no. 3. — P. 211–252.
- [17] Ivanov O. V., Ignatov M. S. On the leaf cell measurements in mosses // *Arctoa*. — 2011. — Vol. 20. — P. 87–98.
- [18] Jolliffe Ian T. [Principal Component Analysis](#). Springer Series in Statistics. — 2 edition. — New York : Springer, 2002.
- [19] Kuhl Frank P., Giardina Charles R. Elliptic Fourier Features of a Closed Contour // [Computer Graphics and Image Processing](#). — 1982. — Vol. 18, no. 3. — P. 236–258.
- [20] PyTorch: An Imperative Style, High-Performance Deep Learning Library / Adam Paszke, Sam Gross, Francisco Massa et al. // *Advances*

- in Neural Information Processing Systems 32 (NeurIPS 2019). — Curran Associates, Inc., 2019. — P. 8024–8035.
- [21] Rohlf F. J. tpsDIG2, version 2.16 [software]. — New York: Stony Brook University, Department of Ecology and Evolution. — 2021.
 - [22] Ronneberger Olaf, Fischer Philipp, Brox Thomas. [U-Net: Convolutional Networks for Biomedical Image Segmentation](#) // Medical Image Computing and Computer-Assisted Intervention (MICCAI). — 2015. — P. 234–241.
 - [23] Rydin Håkan, Gunnarsson Ulf, Sundberg Sebastian. [The role of Sphagnum in peatland development and persistence](#) // Boreal Peatland Ecosystems / Ed. by R. K. Wieder, D. H. Vitt. — Heidelberg : Springer, 2006. — Vol. 188 of Ecological Studies. — P. 47–65.
 - [24] Scikit-learn: Machine Learning in Python / Fabian Pedregosa, Gaël Varoquaux, Alexandre Gramfort et al. // Journal of Machine Learning Research. — 2011. — Vol. 12. — P. 2825–2830.
 - [25] [Searching for MobileNetV3](#) / Andrew Howard, Mark Sandler, Grace Chu et al. // Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV). — 2019. — P. 1314–1324.
 - [26] [Segment Anything](#) / Alexander Kirillov, Eric Mintun, Nikhila Ravi et al. // Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV). — 2023. — P. 4015–4026.
 - [27] Tan Mingxing, Le Quoc V. EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks // Proceedings of the 36th International Conference on Machine Learning (ICML). — Vol. 97 of Proceedings of Machine Learning Research. — PMLR, 2019. — P. 6105–6114.
 - [28] A versatile phenotyping system and analytics platform reveals diverse temporal responses to water availability in Setaria / Noah Fahlgren, Malia Feldman, Malia A. Gehan et al. // [Molecular Plant](#). — 2015. — Vol. 8, no. 10. — P. 1520–1535.